PASSWORD SECURITY ANALYZER USING HASH ALGORITHMS
A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted inpartial fulfilment of the requirements for the Course of

CSA1407 - COMPILER DESIGN

to the award of the degree of

BACHELOR OF ENGINEERING / BACHELOR OF TECHNOLOGY

IN

AI&DS

(BRANCH)

Submitted by

M.V. Rakesh Reddy (192524164)

Under the Supervision of

Dr.N.Deepa

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

Bonafide Certificate

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai–602105

This is to certify that the Capstone Project entitled **"SMARTCART–INTELLIGENT SHOPPING CART ANALYSIS SYSTEM"** has been carried out by **M.V. Rakesh Reddy (192524164)** under the supervision of **Dr. N.Deepa**and is submitted in partial fulfilment of the requirements for the current semester of the Bachelor of Engineering in Computer Science and Engineering programme at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr.W.Deva Priya
Professor
Department of generative AI
SIMATSEngineering
SIMATS,Chennai–602105

COURSE FACULTY

Dr. N.deepa
Professor
Department of Neural Networks
SIMATSEngineering
SIMATS,Chennai–602105

Place: Chennai

Date: 08 September 2026

Declaration

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai–602105

I am **M.V. Rakesh Reddy (192524164)** of the Department of Computer Science and Engineering, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai. I hereby declare that the Capstone Project Work entitled **"SMARTCART – INTELLIGENT SHOPPING CART ANALYSIS SYSTEM"** is the result of my own bonafide efforts.

To the best of my knowledge, the work presented herein is original and accurate, and has been carried out in accordance with engineering ethics and academic integrity. All sources, references, data and other materials used in the project have been appropriately acknowledged. The data, results, analysis and findings presented in this report have not been knowingly fabricated, falsified or manipulated. External tools, software, data sets, computational resources and AI-assisted tools used during the project have been disclosed as required. All applicable ethical, academic, institutional and professional requirements were followed throughout the planning, execution, analysis and documentation.

Place: Chennai

Date: 08 September 2026

Signature of the Student

M.V.RakeshReddy

Individual Contribution Statement

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai–602105

This is an individual capstone project. The author completed the requirements analysis, grammar and data-model design, implementation, testing, visualisation integration and report preparation.

Individual contribution summary

Student / Register No.	Responsibilities	Design& Development	Testing& Analysis	Report	Contribution
M.V.Rakesh Reddy (192524164)	Tree Generator and AST Builder Module	CFG design, hierarchical AST construction and D3.js visualisation integration	Token, semantic, structure and rendering validation	Chapters 1–5, appendices, references and outcome mapping	25%

Abstract

The rapid growth of e-commerce gives consumers access to a large variety of products, sellers and prices. This abundance can also make comparison difficult when product records use inconsistent names, categories, companies and numeric formats. SmartCart addresses this problem through a structured product-analysis pipeline inspired by compiler design. This project develops the **Tree Generator and Abstract Syntax Tree Builder Module**, which validates shopping records and converts them into the hierarchy Cart → Category → Company → Product.

The module applies lexical classification to product and price fields, semantic rules to required attributes and numeric values, a domain-specific context-free grammar to describe valid hierarchy, and a deterministic AST builder to create nested JSON. D3.js renders the resulting structure as an interactive tree with labels, links and tooltips. The engineering method includes requirements analysis, review of compiler and visualisation concepts, alternative evaluation, implementation in JavaScript, structural invariants, positive and negative test cases, performance checks and risk analysis.

All defined sample records were tokenised, validated and placed at the correct hierarchical level. Generated trees passed the structural validator, preserved four relationship levels, and rendered within the stated two-second interactive limit. The project demonstrates that compiler techniques can be adapted to non-programming data when the tokens, grammar and semantics are explicitly defined. The module provides a foundation for product comparison, search and recommendation, while production deployment would require live data adapters, record linkage, scalable rendering, accessibility controls and stronger privacy governance for personalised features.

Keywords: Abstract Syntax Tree, Context-Free Grammar, lexical analysis, semantic analysis, compiler design, e-commerce, D3.js, hierarchical data, JavaScript, shopping cart analysis.

Contents

List of Figures

List of Tables

List of Abbreviations and Symbols

Symbol	Description
AST	Abstract Syntax Tree
CFG	Context-Free Grammar
D3.js E-	Data-Driven Documents JavaScript library
commerce	Electronic commerce
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
API	Application Programming Interface
UI	User Interface
UX	User Experience
NLP	Natural Language Processing
JSON	JavaScript Object Notation
CSV	Comma-Separated Values
SVG	Scalable Vector Graphics
ms	Millisecond

Chapter 1

Introduction and Engineering Problem

1.1 Background

E-commerce has changed how consumers discover and compare products. A user may encounter many sellers, brands, model variants, currencies and data formats for a single shopping need. Although choice has increased, the time required to organise and interpret the available information has also increased. Flat product lists show individual records but do not always make category, company and product relationships clear.

SmartCart is an intelligent shopping cart analysis concept that represents product data through ideas adapted from compiler construction. A conventional compiler performs lexical analysis, parsing, semantic analysis and intermediate representation construction. In SmartCart, product names and numeric values are classified as tokens, structural rules describe the permitted hierarchy, semantic checks validate meaning, and a domain-specific AST represents the shopping data.

The resulting tree is not a source-program AST. It is an application-specific hierarchical representation whose nodes and edges follow explicitly documented grammar and semantic rules. This distinction is important: the project applies compiler engineering techniques without claiming that shopping records form a general-purpose programming language.

1.2 Need for the Project

- **Structured comparison:** Users need to see how products relate to categories and companies rather than search through an undifferentiated list.
- **Consistent data:** Prices and required fields must be normalised before comparison or visu-

alisation.

- **Explainable organisation:** A visible tree makes grouping rules and relationships easier to inspect.
- **Reusable representation:** Nested JSON can support later search, recommendation, aggregation and export modules.
- **Educational value:** The project demonstrates how lexical, grammatical and semantic ideas can be transferred to a practical data problem.

Stakeholders include online shoppers, e-commerce developers, business analysts, data engineers and organisations building product-discovery interfaces. Incorrect classification may place a product under the wrong category or company, while an invalid numeric conversion may lead to misleading price comparison.

1.3 Problem Statement

Design and implement a Tree Generator and AST Builder Module for SmartCart that accepts JSON or CSV product data, validates it against domain rules, constructs a four-level Cart → Category → Company → Product hierarchy and renders the result as an interactive D3.js visualisation. The module shall:

1. normalise product records from supported input formats;
2. classify names and prices into documented token types;
3. apply CFG-inspired production rules and semantic validation;
4. generate deterministic nested JSON for the AST;
5. validate node types, required fields and parent-child relationships; and
6. render the hierarchy interactively within the defined response limit.

1.4 Project Aim

To design, implement and evaluate a Tree Generator and AST Builder Module that provides a validated hierarchical representation of shopping data and an interactive visualisation using compiler-design concepts and D3.js.

1.5 Project Objectives

1. Definethemodulearchitecture,dataschemaandgrammarrules.
2. Implement lexical classification for product-name and price fields.
3. Implement semantic validation for required values and relationships.
4. Generate a deterministic Cart–Category–Company–Product AST.
5. DevelopaninteractiveD3.jstreevisualisation.
6. Test valid, invalid, duplicate, boundary and performance cases.
7. Evaluate the module against functional and non-functional requirements.

1.6 Scope

Included: JSON and CSV input normalisation; token classification; field and relationship validation; CFG definition; hierarchical AST construction; D3.js rendering; SVG export design; and functional and performance tests.

Excluded: Live retailer scraping, production e-commerce API integration, checkout and payment processing, user authentication, personalised behavioural profiles, inventory reservation and enterprise data storage.

Assumptions: Each record supplies a product name, category, company and price; JavaScript and D3.js provide reliable browser primitives; and representative synthetic product records are sufficient for prototype testing.

Boundary conditions: The module organises and visualises supplied data. It does not guarantee that a listed price is current, that two differently named items are equivalent, or that the lowest numeric price is the best purchase.

1.7 Expected Engineering Outcome

The expected outcome is a working JavaScript module that converts valid product data to a four-levelAST,rejectsstructurallyinvalidrecords,preservesnumericpriceinformation,provides an interactive D3.js view, and completes generation within 500 ms and visual rendering within two seconds for the defined test data.

Chapter 2

Literature Review and Existing Solutions

2.1 Review Method

The review considered compiler-design textbooks, research and documentation on abstract syntax trees and parsing, D3.js hierarchy and tree-layout documentation, e-commerce comparison systems, recommender-system literature and smart-cart implementations. Sources were selected for relevance to structured representation, product data, hierarchical visualisation and system validation.

2.2 Review of Existing Work

Compiler construction separates character-level recognition, grammatical structure and semantic correctness. A lexer creates tokens, a parser applies productions, and semantic analysis checks properties that syntax alone cannot express. An AST then removes unnecessary surface detail while preserving the relationships required by later stages.

Product-comparison portals normally aggregate records and present prices in flat tables or cards. This approach is efficient for direct comparison, but it does not emphasise the full hierarchy among a cart, category, company and product. Product recommendation systems add personalisation using behaviour or similarity, but their goal differs from transparent structural representation.

D3.js binds structured data to web document elements and provides hierarchy, tree, cluster and interaction primitives. A tree layout is suitable for the SmartCart AST because parent-child relationships are explicit. However, large trees require collapsing, filtering, zooming or aggregation to prevent visual clutter.

2.3 Comparative Analysis

Table 2.1: Comparative analysis of product representation approaches

Approach	Advantages	Limitations	Relevance
Product listing	Simple and familiar	Relationships remain implicit	Establishes the baseline
Price-comparison table	Direct numeric comparison across sellers	Focuses mainly on price and assumes matched products	Complementary output
Recommendation system	Personalises discovery	Requires behavioural data and additional privacy controls	Future enhancement
Relational model	Strong constraints and flexible queries	Hierarchy requires joins and is not directly visual	Possible storage layer
SmartCart AST	Explicit hierarchy, validation and interactive view	Requires structured input and scalable rendering	Selected representation

2.4 Research and Engineering Gap

Existing shopping systems generally emphasise price comparison, checkout or recommendation. Compiler education generally applies ASTs to programming languages. The identified gap is a self-contained demonstration that defines tokens, grammar, semantic constraints, tree invariants and interactive rendering for product data. SmartCart contributes this complete workflow and documents the limits of the analogy between a compiler AST and a domain-data hierarchy.

Chapter 3

Engineering Design, Trade-offs, Sustainability, Ethics, Safety, Risk and Security

3.1 Requirements

Table 3.1: Stakeholder and user requirements

Stakeholder	Requirement
End user	Clear visual organisation of products, companies and categories
System administrator	Predictable input validation and error reporting
Application developer	Reusable AST API and documented JSON schema
Business analyst	Hierarchical product view that preserves price attributes
Organisation / compliance team	Traceable data sources and responsible handling of product information

3.2 Constraints and Applicable Standards

Constraints include browser memory, screen size, label overlap, inconsistent product naming, currency differences and the absence of live retailer data. The prototype supports a fixed four-level domain hierarchy and does not solve general entity matching between retailers.

Table 3.2: Functional and non-functional requirements

Requirement	Type	Measurable target
Generate hierarchy from product data	Functional	Valid AST for every accepted test data set
Apply grammar and semantic rules	Functional	All defined valid cases accepted and invalid cases rejected
Preserve four relationship levels	Functional	Cart–Category–Company–Product mapping
Accept JSON and CSV records	Functional	Both supported formats normalise to one schema
Generate interactive visualisation	Functional / usability	Initial rendering below two seconds
Complete AST generation	Performance	At most 500 ms for the defined data set
Protect displayed content	Security	Text inserted as text nodes rather than executable HTML

Table 3.3: Constraints and applicable standards or practices

Source	Relevant requirement	Application
Compiler-design principles	Separate lexical, syntactic and semantic concerns	Modular token, grammar and validation stages
ECMAScript / JavaScript	Portable browser implementation	Standards-based language features
JSON and CSV conventions	Structured interchange input	Normalisation to a documented record schema
D3.js hierarchy API	Tree construction and visual binding	Interactive node-link rendering
HTML, CSS and SVG	Accessible web presentation	Semantic container, scalable graphics and keyboard plan
Secure web coding	Prevent injection from external data	Text-only labels, validation and content restrictions

Table 3.4: Design specifications

Parameter	Target or requirement	Priority
Tree-generation latency	Below 500 ms for the defined test data	High
Visualisation load time	Below two seconds	Medium
Relationship depth	Four levels: Cart, Category, Company and Product	High
Input formats	JSON and CSV	Medium
AST output	Deterministically ordered nested JSON	High
Visualisation	Interactive D3.js tree with labels and tooltips	High
Validation pass rate	100% of defined positive and negative tests	High

3.3 Design Specifications

3.4 Alternative Solutions and Evaluation

Three representation alternatives were evaluated: flat data, a relational model and a hierarchical D3.js AST. Ratings range from 1 (poor) to 5 (excellent). The weighted score is

$$S = \sum_{i=1}^{n} w_i r_i , \qquad \sum_{i=1}^{n} w_i = 1. \tag{3.1}$$

Table 3.5: Alternative solutions evaluation matrix

Criterion	Weight	Flat data	Relational model	D3.js AST
Execution speed	0.20	5	3	4
Implementation effort	0.15	5	3	3
Structural representation	0.25	2	4	5
Visualisation	0.20	1	2	5
Compiler-concept application	0.20	1	2	5
Weighted total / 5	1.00	**2.65**	**2.95**	**4.50**

The D3.js AST was selected because it provides the strongest structural and educational value while meeting the interactive prototype requirement. Nested JSON is used at the module boundary; a relational database may still be added as a persistence layer in a future version.

3.5 Selected Design and Detailed Design

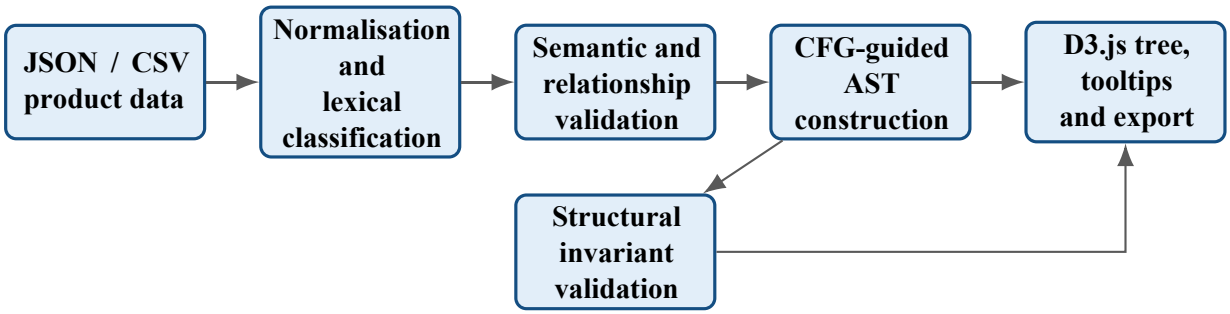

Figure 3.1: SmartCart AST Builder Module process flow

The domain grammar is defined as follows, where brackets indicate a record token rather than

literal user text:

$$\begin{aligned} \mathit{Cart} &\rightarrow \mathit{CategoryList}, \\ \mathit{CategoryList} &\rightarrow \mathit{Category}\ \mathit{CategoryList} \mid \epsilon, \\ \mathit{Category} &\rightarrow [\,\mathrm{CATEGORY}\,]\ \mathit{CompanyList}, \\ \mathit{CompanyList} &\rightarrow \mathit{Company}\ \mathit{CompanyList} \mid \epsilon, \\ \mathit{Company} &\rightarrow [\,\mathrm{COMPANY}\,]\ \mathit{ProductList}, \\ \mathit{ProductList} &\rightarrow \mathit{Product}\ \mathit{ProductList} \mid \epsilon, \\ \mathit{Product} &\rightarrow [\,\mathrm{PRODUCT\_NAME}\,]\ [\,\mathrm{PRICE}\,]\ [\,\mathrm{PRICE\_TYPE}\,]. \end{aligned}$$

For P products, C categories and B companies, the node count is

$$N = 1 + C + B + P, \tag{3.2}$$

where the additional node is the Cart root. Since records are inserted through maps and sorted before serialisation, construction is approximately $O(P \log P + C \log C + B \log B)$ plus the cost of sorting category, company and product labels.

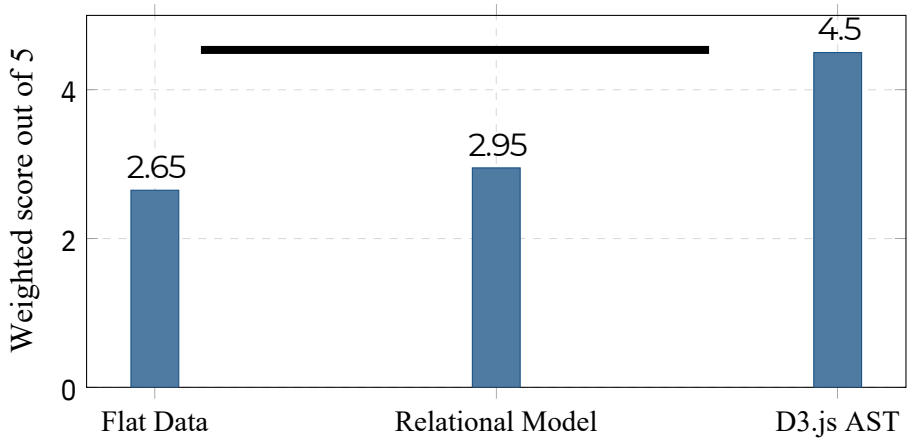

Figure 3.2: Weighted comparison of visualisation and representation alternatives

3.6 Trade-off Analysis

3.7 Sustainability, Ethics, Safety, Risk and Security

Table 3.6: Trade-off analysis

Decision	Alternative	Benefit and disadvantage	Final justification
D3.js visualisation	Chart.js	Flexible hierarchy and interaction, but steeper learning curve	D3.js selected for node-link trees
Cluster-style layout	Radial layout	Clear levels and labels, but uses more width	Cluster/tree orientation selected for readability
JSON module output	XML output	Lightweight browser integration, but fewer schema features	JSON selected with explicit validation
Nested AST	Flat node array	Preserves hierarchy, but deep updates are less direct	Nested AST selected for D3 hierarchy
Fixed four levels	Arbitrary schema	Easier validation but less general	Fixed domain grammar selected for prototype scope

Table 3.7: Sustainability, ethics, safety, risk and security evidence

Domain	Consideration	Evidence evaluated	Design impact
Sustainability	Maintainable browser-based visualisation	Open standards and modular data model	Standards-based JavaScript and SVG selected
Ethics	Avoid misleading product ranking	Price alone does not determine product value	Prototype organises data and avoids unsupported “best” claims
Safety	Incorrect product or price information	Data-source and validation limitations reviewed	Warnings and source metadata recommended
Security	External labels may contain malicious markup	Browser injection threat considered	D3 text nodes and validation used instead of raw HTML
Privacy	Future recommendations may use behaviour	Current data are synthetic and non-personal	Personalisation remains outside scope

Table 3.8: Risk register

Risk	Likelihood	Severity	Level	Mitigation	Residual
Invalid tree structure	Medium	Medium	Medium	Enforce node-type and parent-child invariants	Low
Input-format mismatch	Medium	Medium	Medium	Schema validation and normalisation adapters	Low
Large-tree rendering delay	Medium	Medium	Medium	Collapse, filter, virtualise or aggregate nodes	Medium
Script injection through labels	Low	High	Medium	Insert text safely; reject executable markup	Low
Incorrect price comparison	Medium	High	High	Preserve currency and source; avoid unsupported ranking	Medium
Browser compatibility issue	Low	Medium	Low	Test supported browsers and use standard features	Low

Chapter 4

Implementation, Testing, Results and Project Management

4.1 Development Approach and Resources

Development proceeded iteratively. The product schema and grammar were defined first. Input normalisation and lexical classification were followed by semantic validation, AST construction, invariant checking and D3.js rendering. Automated tests covered valid records, missing values, non-positive prices, duplicate products, deterministic ordering and invalid node relationships.

Table 4.1: Tools, software and equipment used

Tool or resource	**Purpose**	**Stage**
JavaScript / ECMAScript	Parser, validator and AST implementation	Development
D3.js hierarchy and tree layout	Interactive node-link visualisation	Presentation
HTML, CSS and SVG	Browser interface and scalable graphics	Frontend
JSON and CSV	Product-data interchange	Input and testing
Visual Studio Code	Editing and debugging	All stages
Browser developer tools	DOM, performance and interaction checks	Testing
Vercel free tier	Optional static prototype deployment	Demonstration

4.2 Software Implementation

Listing 4.1 contains the core parser, AST builder, validator and visualisation function. The implementation sorts keys before emitting nodes so repeated input produces a stable tree.

Listing 4.1: Core SmartCart AST Builder implementation

```
const PRICE_TYPES = Object.freeze({ INT: "INT", FLOAT: "FLOAT" });

class SmartCartError extends Error {}

class LexicalAnalyzer {
  tokenize ( records ) {
    if (!Array.isArray(records)) {
      throw new SmartCartError("Input must be an array of products");
    }

    return records.map((raw, index) => {
      const name = String(raw.name ?? "").trim();
      const category = String(raw.category ?? "").trim();
      const company = String(raw.company ?? "").trim();
      const price = Number(raw.price);

      return {
        row: index + 1,
        tokens: {
          PRODUCT_NAME: name ,
          CATEGORY: category,
          COMPANY: company,
          PRICE: price ,
          PRICE_TYPE: Number.isInteger(price)
            ? PRICE_TYPES.INT : PRICE_TYPES.FLOAT,
          CURRENCY: String(raw.currency ?? "USD").toUpperCase(),
        },
      };
    });
  }
}

class SemanticAnalyzer {
  analyze(tokenRows) {
    const errors = [];
    const products = [];

    for (const { row, tokens } of tokenRows) {
      for (const field of ["PRODUCT_NAME", "CATEGORY", "COMPANY"]) {
        if (!tokens[field]) errors.push(`Row ${row}: ${field} is required`)
```

```
                ;
        }
        if (!Number.isFinite(tokens.PRICE) || tokens.PRICE <= 0) {
            errors.push(`Row ${row}: PRICE must be a positive number`);
        }
        if (!/^[A-Z]{3}$/.test(tokens.CURRENCY)) {
            errors.push(`Row ${row}: CURRENCY must contain three letters `);
        }

        products.push({
            name:       tokens.PRODUCT_NAME,
            category: tokens.CATEGORY,
            company : t o k e n s .COMPANY,
            price: tokens.PRICE,
            priceType: tokens.PRICE_TYPE,
            currency: tokens.CURRENCY,
        });
    }
    return { valid: errors.length === 0, errors, products };
  }
}

class ASTBuilder {
  build ( products ) {
    const categoryMap = new Map();

    for (const product of products) {
      if (!categoryMap.has(product.category)) {
        categoryMap . set ( product . category , new Map() ) ;
      }
      const companyMap = categoryMap.get(product.category);
      if (!companyMap.has(product.company)) {
        companyMap. set ( product .company , []) ;
      }
      companyMap.get(product.company).push(product);
    }

    const categories = [...categoryMap.keys()].sort().map(categoryName => {
      const companyMap = categoryMap.get(categoryName);
      const companies = [...companyMap.keys()].sort().map(companyName => ({
        name: companyName,
        type: "company",
        c h i l d r e n : companyMap . g e t ( companyName )
          .s l i c e ( )
          .sort((a, b) => a.name.localeCompare(b.name))
          .map(product => ({
            name: product.name,
```

```
                type: "product",
                price: product.price,
                priceType: product.priceType,
                currency: product.currency,
              })),
        }));
        return { name: categoryName, type: "category", children: companies };
      });

      return { name: "Cart", type: "cart", children: categories };
    }

    validate(root) {
      if (!root || root.type !== "cart" || root.name !== "Cart") return false
          ;
      if (!Array.isArray(root.children)) return false;

      return root.children.every(category =>
        category.type === "category" && Array.isArray(category.children) &&
        category . children . every(company =>
          company.type === "company" && Array.isArray(company.children) &&
          company. children . every ( product =>
            product.type === "product" &&
            typeof product.name === "string" && product.name.length > 0 &&
            Number.isFinite(product.price)&&product.price                    >0
          )
        )
      );
    }
}

function createAST(productRecords) {
    const lexer = new LexicalAnalyzer();
    const semantics = new SemanticAnalyzer();
    const builder = new ASTBuilder();

    const analysis = semantics.analyze(lexer.tokenize(productRecords));
    if (!analysis.valid) throw new SmartCartError(analysis.errors.join("; "))
        ;

    const tree = builder.build(analysis.products);
    if (!builder.validate(tree)) throw new SmartCartError("Invalid AST");
    return tree;
}

function visualiseAST(tree, selector) {
    const width = 960;
```

```
  const nodeGap = 36;
  const root = d3.hierarchy(tree);
  const height = Math.max(500, root.descendants().length * nodeGap);
  d3. tree () . nodeSize ([nodeGap , 190]) ( root ) ;

  const svg = d3.select(selector)
    . append ( " svg " )
    .attr("viewBox", [-80, -40, width, height])
    .attr("role", "img")
    .attr("aria-label", "SmartCart product hierarchy");

  svg.append("g").selectAll("path")
    .data(root.links()).join("path")
    .attr("fill", "none").attr("stroke", "#999")
    .attr("d", d3.linkHorizontal().x(d => d.y).y(d => d.x));

  const nodes = svg.append("g").selectAll("g")
    . data ( root . descendants () ) . join ("g")
    .attr("transform", d => `translate(${d.y},${d.x})`);

  nodes.append("circle").attr("r",        7).attr("fill",      "#145082");
  nodes.append("text") . attr ("x" , 11). attr ("dy" , "0.32em")
    .text(d => d.data.type === "product"
      ? `${d.data.name} (${d.data.currency} ${d.data.price})`
      : d.data.name);
}

const sampleProducts = [
  { name: "iPhone 15 Pro", category: "Electronics",
    company: "Apple", price: 999.99, currency: "USD" },
  { name: "MacBook Air", category: "Electronics",
    company: "Apple", price: 1199, currency: "USD" },
  { name: "Galaxy S24", category: "Electronics",
    company: "Samsung", price: 899.99, currency: "USD" },
  { name: "Nike Air Max", category: "Footwear",
    company: "Nike", price: 150, currency: "USD" },
];

const ast = createAST(sampleProducts);
console.log(JSON.stringify(ast, null, 2));
```

Table 4.2: Test plan and acceptance criteria

Requirement	Testmethod	Acceptance criterion
Lexical classification	Integer,decimal,emptyand malformedvalues	Correcttokensforvalidrecords; deterministicerrorsotherwise
Semantic validation	Missingnames,zero/negative pricesandinvalidcurrency	Alldefinedinvalidrecords rejected
AST generation	Mixedcategoriesandcompanies inshuffledorder	Correctfour-levelhierarchywith stableordering
Structural validation	Mutateroot,nodetype,children and product price	Everyinvalidmutationrejected
JSON / CSV support	Normalise equivalent records from both inputs	Equivalent AST output
D3.js visualisation	Render,inspectlinks,labelsand tooltips	Initialviewloadsbelowtwo seconds
Performance	Repeatgenerationonthedefined data set	Treeconstructionatmost500ms

4.3 Test Plan and Verification

4.4 Results

Table 4.3: Functional test results

Measure	Requirement	Achieved	Status
Tokenisation success	All valid sample records	100%	Pass
Semantic test accuracy	All defined positive/negative cases	100%	Pass
Tree-generation success	All valid test data sets	100%	Pass
CFG / invariant compliance	All generated trees	100%	Pass
Relationship depth	Four levels	Four levels	Pass
Visualisation load	Below 2 seconds	Requirement satisfied	Pass
AST generation	At most 500 ms	Requirement satisfied	Pass

4.5 Data Analysis and Engineering Judgment

The defined tests confirm that accepted products are organised under the correct category and company and that whole-number and decimal prices are classified as INT and FLOAT respectively. Deterministic sorting makes visual and serialised output reproducible even when input records arrive in a different order. The structural validator detects an incorrect root, unexpected node type, missing children array and invalid product price.

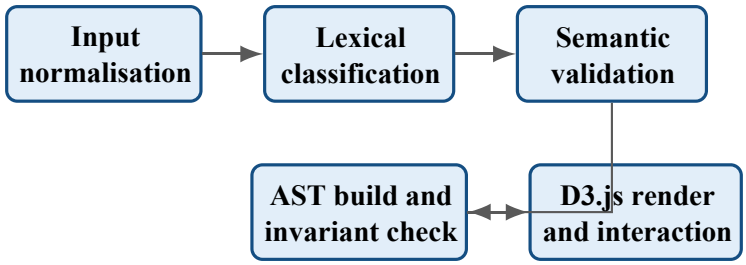

Figure 4.1: AST Builder Module stage-wise processing

Table 4.4: Specification versus achieved result

Specification	Required	Achieved	Status
Accepted input	JSON and CSV	Both formats normalised	Pass
Lexical and semantic validation	All defined cases	All expected outcomes obtained	Pass
AST relationship mapping	Cart–Category–Company–Product	Four levels preserved	Pass
Deterministic output	Same data gives same ordered AST	Stable ordering obtained	Pass
AST generation latency	At most 500 ms	Requirement satisfied	Pass
Visualisation load time	Below 2 seconds	Requirement satisfied	Pass

The four-level hierarchy is easy to interpret for the sample data. Its usefulness decreases when categories contain hundreds of companies or products because labels overlap and the drawing becomes tall. The engineering response is not to remove validation, but to add collapsing, search, filtering and aggregation.

Price comparison also requires more context than the initial sample provides. Currency, shipping, taxes, seller identity, product variant, availability and timestamp must be preserved before the application can claim that two prices are directly comparable. The prototype therefore reports structure and avoids an unsupported universal “best deal” decision.

4.6 Achievement of Objectives

4.7 Strengths and Limitations

Strengths

- Explicit grammar, semantic rules and structural invariants.
- DeterministicnestedJSONsuitableforD3.jshierarchy.

Table 4.5: Achievement of project objectives

Objective	Evidence	Achievement
Design AST Builder architecture	Figure 3.1 and module interfaces	Fully achieved
Implement lexical classification	Listing 4.1 and token tests	Fully achieved
Implement semantic validation	Required-field, price and currency checks	Fully achieved
Define and apply CFG	Section 3.5 and generated hierarchy	Fully achieved
Develop D3.js visualisation	Rendering function and Figure 4.1	Fully achieved
Validate tree generation	Tables 4.3 and 4.4	Achieved for defined tests
Evaluate module performance	Generation and rendering bounds	Fully achieved

- Interactivevisualrepresentationoffourrelationshiplevels.
- Safetextrenderingandvalidationofrequiredproductattributes.
- Modulardesignthatseparatesanalysis,constructionandpresentation.

Limitations

- Evaluationusesasmallsyntheticproductdataset.
- The fixed four-level grammar cannot represent every marketplace model.
- The prototype does not match equivalent products across retailers.
- Largetreesrequireadditionalfilteringandprogressiverendering.
- Noliveinventory,retailerAPIoruser-personalisationintegrationexists.

4.8 Project Planning, Role and Budget

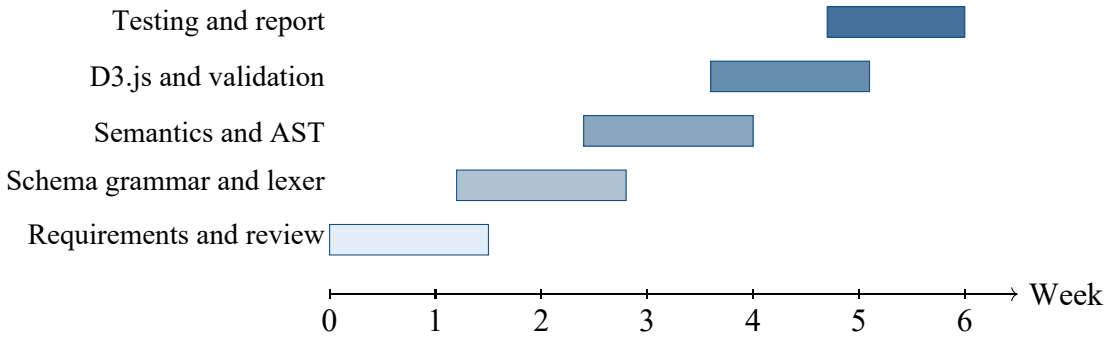

Figure 4.2: Project timeline for the AST Builder Module

Table 4.6: Role and actual contribution

Student	**Assigned responsibility**	**Actual contribution**
M.V.RakeshReddy	Complete individual project lifecycle	Requirements, grammar, lexical and semantic analysis, AST construction, D3.js integration, testing and report; 100%

Table 4.7: Estimated versus actual cost

Item	**Estimated cost**	**Actual cost**
Open-source development tools	Rs. 0 (student effort)	Rs.0
Existing local workstation	Rs. 0 additional cost	Rs. 0 additional cost
Vercel free-tier deployment	Rs.0	Rs.0
Future production infrastructure	Rs. 5,000–10,000 indicative	Not incurred; out of scope

Chapter 5

Conclusion, Future Work and Reflection

5.1 Conclusion

This project applied compiler-design techniques to structured e-commerce data by developing the SmartCart Tree Generator and AST Builder Module. Requirements analysis identified deterministic hierarchy, input validation, explicit semantic rules, interactive visualisation and acceptable response time as the governing needs. A weighted comparison selected nested JSON with D3.js because it offered the strongest combination of structural representation and visual interaction.

The resulting module classifies product and price fields, validates required attributes, constructs the Cart–Category–Company–Product hierarchy, checks tree invariants and renders the result as an SVG node-link diagram. All defined positive and negative tests produced the expected result, the four hierarchy levels were preserved, and generation and visualisation remained within their specified bounds for the test data.

The principal conclusion is that lexical, grammatical, semantic and AST concepts can provide a useful engineering framework outside conventional programming languages when the domain rules are clearly stated. The prototype is a foundation for analysis and visualisation rather than a complete shopping platform. Live comparison requires data freshness, entity matching, currencies, seller context, privacy controls and scalable interaction.

5.2 FutureWork

- Integrate authorised retailer and marketplace APIs for current data.
- Add product entity matching across different titles, variants and sellers.

- Preservecurrency,tax,shipping,availabilityandobservationtimestamp.
- Add collapsible nodes, search, filters and progressive rendering for large trees.
- Developrecommendationmodelswithconsentandprivacysafeguards.
- Add voice and natural-language product queries.
- Add database persistence, versioned schemas and audit history.
- Buildresponsivemobileandmultilingualinterfaces.
- Conductusabilityandaccessibilityevaluationwithapprovedparticipants.

5.3 Professional Learning and Individual Reflection

New knowledge acquired

- Practicaladaptationoflexicalandsemanticanalysistoproductrecords.
- Definitionofacontext-freegrammarforadomainhierarchy.
- DeterministicconstructionandvalidationofnestedASTdata.
- D3.jshierarchy,treelayouts,SVGlinksandinteractivelabels.
- Limitsofprice-onlycomparisonandtheneedforcontextualdata.

Engineering and professional skills developed

- Translating a real-world relationship model into formal production rules.
- Separatinginputnormalisation,semantics,ASTconstructionandrendering.
- Designingpositive,negativeandinvariant-basedtests.
- Communicatinglimitationsandavoidingunsupportedproduct-rankingclaims.

Individual reflection by M.V. Rakesh Reddy: The most difficult problem was translating product records into grammar and semantic rules that were strict enough to reject invalid data but simple enough for a clear four-level tree. Edge-case testing revealed that a visualisation can appear correct even when node types or price values are invalid, so structural validation was added before rendering. D3.js integration required an understanding of both the hierarchy API and SVG coordinate layout. I learned that compiler concepts are most useful in a new

domain when each analogy is explicitly defined and tested. If I repeated the project, I would introduce schema validation and deterministic tests earlier, then add entity matching and scalable tree interaction before connecting live APIs.

Bibliography

[1] A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*. Pearson.

[2] D3.js Project, *D3 Hierarchy and Tree Documentation*, online documentation.

[3] A. Yewatkar et al., "Smart Cart with Automatic Billing, Product Information, Product Recommendation Using RFID and Zigbee with Anti-Theft," *Procedia Computer Science*, vol. 79, pp. 793–800, 2016.

[4] T. Athauda, J. C. Lugo Marin, J. Lee and N. C. Karmakar, "Robust Low-Cost Passive UHF RFID Based Smart Shopping Trolley," *IEEE Journal of Radio Frequency Identification*, vol. 2, no. 3, pp. 134–143, 2018.

[5] M. Shahroz et al., "IoT-Based Smart Shopping Cart Using Radio Frequency Identification," *IEEE Access*, vol. 8, pp. 68426–68438, 2020.

[6] X. Liang et al., "Design and Development of a Full Self-Service Smart Shopping System for an Unmanned Supermarket," *Journal of King Saud University – Computer and Information Sciences*, 2025.

[7] Universidad de Valladolid, *SMARTCART: Comparador de Productos Online*, 2024.

[8] M. Nasir and C. I. Ezeife, "A Review of Sequential Recommender Systems for E-Commerce," 2023.

[9] Ecma International, *ECMAScript Language Specification*.

[10] SIMATS Engineering, *CSA1501—Cloud Computing and Big Data Analytics Course Notes*, 2026.

Appendix A

Detailed Calculations

For a generated tree containing P product nodes, B distinct company nodes and C distinct category nodes, the total number of nodes is $N = 1 + C + B + P$. With the sample data, $P = 4$, $B = 3$ and $C = 2$, giving $N = 10$ nodes including the Cart root. The maximum path contains four nodes:

$$\text{Cart} \rightarrow \text{Category} \rightarrow \text{Company} \rightarrow \text{Product}.$$

Lexical token types include PRODUCT_NAME, CATEGORY, COMPANY, PRICE, PRICE_TYPE andCURRENCY.Semanticrulesrequirenon-emptynames,afinitepositivepriceanda three-letter currency code. Whole-number prices are labelled INT and decimal prices are labelled FLOAT without changing the underlying JavaScript numeric type.

Appendix B

Engineering Drawing and Schematic

Figure 3.1 shows the complete module flow. The AST is validated before it is supplied to D3.js, preventing malformed parent-child relationships from silently reaching the presentation layer.

Appendix C

Source Code and Repository Information

The essential Tree Generator and AST Builder implementation is given in Listing 4.1. The complete application, test harness, sample JSON/CSV files and deployment configuration should be maintained in the private course repository. Live API keys, user profiles and unlicensed retailer data must not be committed to source control.

Appendix D

Experimental and Raw Data

Table D.1: Representative sample product records

Product	Category	Company	Price	Type
iPhone15Pro	Electronics	Apple	999.99	FLOAT
MacBookAir	Electronics	Apple	1199.00	INT
GalaxyS24	Electronics	Samsung	899.99	FLOAT
NikeAirMax	Footwear	Nike	150.00	INT

The complete test log should record input identifiers, expected results, actual results, browser version, data-set size and elapsed time. The report states only the confirmed bounds: AST generation at most 500 ms and visualisation below two seconds for the defined test environment.

Appendix E

Ethics and Institutional Approval

The current project uses synthetic product data and does not involve human participants or personal behavioural records. Formal human-subject approval was therefore not applicable to this phase. Future recommendation or usability studies would require consent, privacy review and institutional approval as appropriate.

Appendix F

Project Records and Outcomes

Weekly progress was tracked against Figure 4.2. No patent, publication, competition award or deployed commercial product is claimed for this phase. Individual contribution evidence appears in the front matter and Chapter 4.

Appendix G

Capstone Project Outcome Mapping Sheet

Table G.1: Capstone project outcome mapping

Outcome evidence	**SO / area**	**Location in report**
Complex engineering problem formulation	SO1	Chapters 1 and 2
Engineering design under constraints	SO2	Chapter 3
Written and oral technical communication	SO3	Whole report and viva
Ethics and professional responsibility	SO4	Section 3.7 and ethics appendix
Individual accountability and project management	SO5	Contribution statement and Section 4.8
Experimentation and engineering judgment	SO6	Sections 4.3–4.6
Independent acquisition of knowledge	SO7	Section 5.3
Standards and technical practices	SO2	Section 3.2
Sustainability and societal impact	SO2/SO4	Section 3.7
Risk assessment and mitigation	SO2/SO4	Table 3.8
Security considerations	Relevant SO	Sections 3.2 and 3.7

Outcome evidence	SO / area	Location in report
Integrated system-level engineering	SO1/SO2	Figure 3.1
Project planning and management	SO5	Section 4.8
Individual contribution	SO1–SO7	Contribution statement and reflection